

Lab 9: Fingerprinting Techniques – Font Identification

In the previous lab, we used Javascript to help randomize values in the *navigator* object, making it harder to take a reliable fingerprint of our browser. However, there are many other techniques for identifying and fingerprinting browsers. In this lab, we'll look at a particular example: identifying fonts installed on the user system.

A browser typically does not advertise what fonts as a system has installed – there is no list of system fonts available as a convenient JavaScript object. However, there is a way to enumerate fonts on a client's machine, through the clever use of JavaScript and CSS rendering.

Getting the Default Text Width

A small starting file has been included with this lab, called *fontfingerprint.html*. Download this file and open it up in a text editor. The contents should look like this:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>Font Identification</title>
7  </head>
8  <body>
9      <p id="test">The quick brown fox jumped over the lazy dog</p>
10 <script>
11     // TODO
12 </script>
13 </body>
14 </html>
```

What our font enumeration script will do is rapidly change the font used to render the *test* paragraph element. We'll check the *width* of the *test* element in between and use a change in width to detect if a font is present.

First, we need a default baseline width, which we'll use to detect when the font is *not* present. First, in our script, we'll get the *test* paragraph element and set the font to *sans-serif*, and the display style to *inline-block*:

```
<script>
// Get the paragraph element and set default styling values
var para = document.getElementById("test");
para.style.fontFamily = "sans-serif";
para.style.display = "inline-block";
```

Next, we'll need to measure the actual width of the element now that we've set the display style and the font. We can do this with the *offsetWidth* property of the paragraph object:

```
// Get width of san-serif line to serve as default width
var default_width = para.offsetWidth;
console.log("Default Text Width: ", default_width);
```

The *default_width* variable will hold the width of a line in the standard *sans-serif* font. This will be the default value that we compare all other fonts against to see if they exist on the host's computer.

Fonts For Identification

Next, we need a list of fonts we want to try to identify. This approach requires that you have a list of fonts you wish to check for; it doesn't allow you to enumerate fonts that aren't on the list. In deciding which fonts should be on there, take some things into account:

- Different operating systems have different default fonts installed; this can give you hints as to the client's OS type
- Some fonts are installed and associated with specific software, and so can provide hints as to what software the host has installed

You should create a list of potential fonts to test for. Below is a list that I came up with myself – it includes various OS-specific fonts (TNR and Segoe are Windows; Lucida Grande and San Francisco are Mac; Droid Sans and Droid Serif are Android) as well as some software-specific fonts as well (ROG Fonts, which will exist if you have certain ASUS firmware installed on your computer):

```
// Now loop through list of potential fonts to test for
const fonts = ["Times New Roman", "Segoe UI", "Lucida Grande", "San Francisco", "Georgia", "Impact",
               "Calibri", "Cambria", "Symbol", "Trebuchet MS", "Webdings", "Verdana", "Consolas", "Droid Sans",
               "Droid Serif", "Cascadia Code", "ROG Fonts"];
```

Your own list may include fonts that I have included, but it should include other fonts as well. You may want to research what kind of software and operating systems install what fonts for this.

Comparing Font Widths

The secret to this font enumeration technique is CSS's default font *fallback* behaviour. Namely, you can specify a list of fonts, and the client will try to render them one by one – if the first font is not available, it will move onto the second font, and so on.

We can use this to check for fonts by listing our “font to test” first and then using *sans-serif* as the second listed font. If the font doesn't exist, the client will default to using *sans-serif*. We then compare the widths of the paragraph and see if it is different than the *san-serif* width. If it is, then we know that the font exists and was rendered; otherwise, the font probably wasn't found.

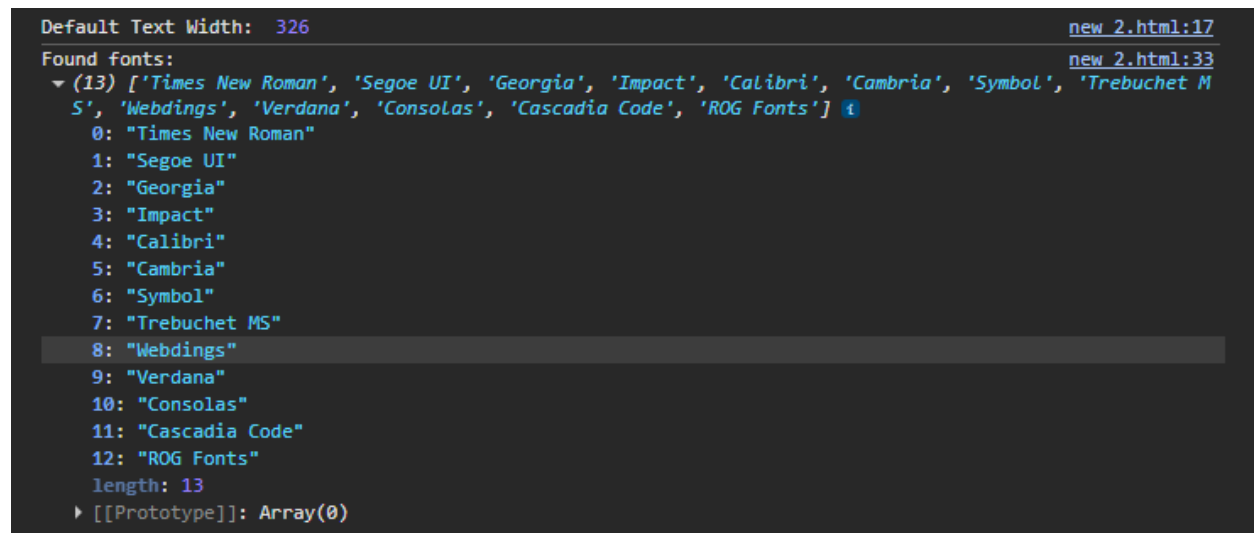
```
const found_fonts = [];  
fonts.forEach(function (font) {  
    // For each font, try the font with sans-serif as the fallback  
    para.style.fontFamily = font + ", sans-serif";  
    var test_width = para.offsetWidth;  
    // Compare text width, if not the same as sans-serif then add to found fonts list  
    if(test_width !== default_width){  
        found_fonts.push(font);  
    }  
});
```

We check the width of the current font against the *default_width* we calculated earlier. If they don't have matching widths (meaning that the font exists in the system and it didn't fall back to sans-serif), then we add the font to the array of found fonts. We do a final printout of the list of found fonts on the client:

```
// Print out the found fonts  
console.log("Found fonts: ", found_fonts);
```

Trying It Out

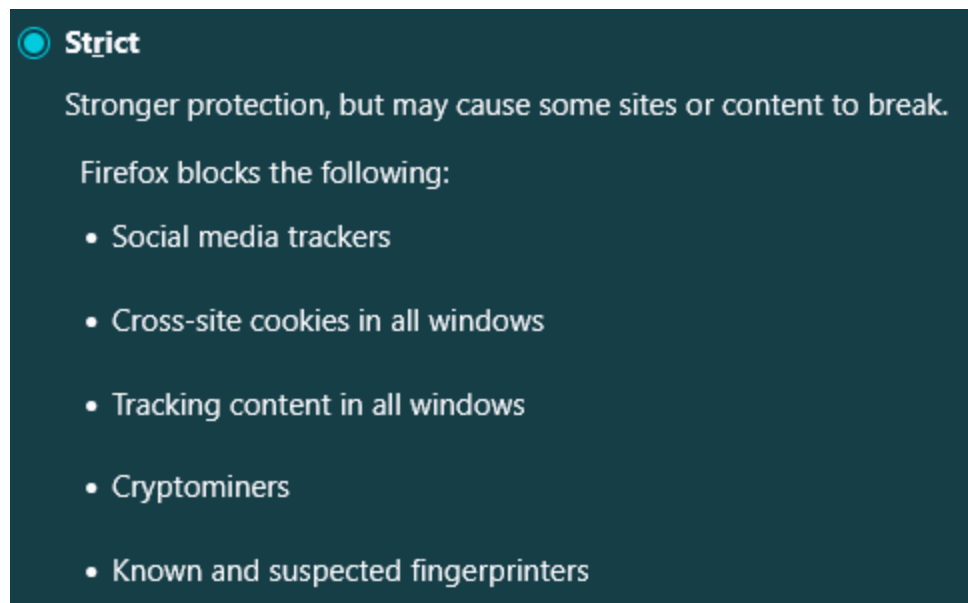
Save this HTML file and try opening it up in **Chrome** or another browser. Right click and open your *inspector*, and go to the *console* to see the output:



```
Default Text Width: 326  
Found fonts:  
▼ (13) ['Times New Roman', 'Segoe UI', 'Georgia', 'Impact', 'Calibri', 'Cambria', 'Symbol', 'Trebuchet M  
S', 'Webdings', 'Verdana', 'Consolas', 'Cascadia Code', 'ROG Fonts']  
0: "Times New Roman"  
1: "Segoe UI"  
2: "Georgia"  
3: "Impact"  
4: "Calibri"  
5: "Cambria"  
6: "Symbol"  
7: "Trebuchet MS"  
8: "Webdings"  
9: "Verdana"  
10: "Consolas"  
11: "Cascadia Code"  
12: "ROG Fonts"  
length: 13  
[[Prototype]]: Array(0)
```

For example, you can tell that I'm probably on a **Windows** machine from this (as I have Windows fonts, but am missing Mac or Android fonts), and you can tell that I have an ASUS graphics card of some sort and have their firmware installed (ROG Fonts was found).

It's also worth mentioning that some browsers have built-in protections designed to prevent you from accessing non-standard fonts. **Firefox** has this capability built in as part of its privacy controls and explicitly advertises that it will attempt to foil common fingerprinters when in "strict" mode:



Trying to run our script in Firefox under the strict privacy settings results in errors for two fonts on our list that should exist on our machine:

```
Default Text Width: 326
⚠ Request for font "Cascadia Code" blocked at visibility level 2 (requires 3)
⚠ Request for font "RDG Fonts" blocked at visibility level 2 (requires 3)
Found fonts:  » Array(11) [ "Times New Roman", "Segoe UI", "Georgia", "Impact", "Calibri", "Cambria", "Symbol", "Trebuchet MS", "Webdings", "Verdana", ... ]
```

This doesn't necessarily provide much protection (as the fact we get an error suggests those fonts exist; so, we can still enumerate them, we just can't use them). However, it does mean that you can't necessarily guarantee the behaviour of this fingerprinter across different browsers, even on the same underlying host with the same installed fonts.

Font fingerprinting is just one example of ways that scripts can extract more information about your system from your browser. There are many other ways of getting your browser to leak information about your system that could be used to uniquely fingerprint you.

Lab Submission

When you are done, submit your completed `fontfingerprint.html` file on Moodle.